

Wstęp do sztucznej inteligencji

Laboratorium 5

Sztuczne sieci neuronowe

Wiktor Trawiński

Kacper Skrodzki

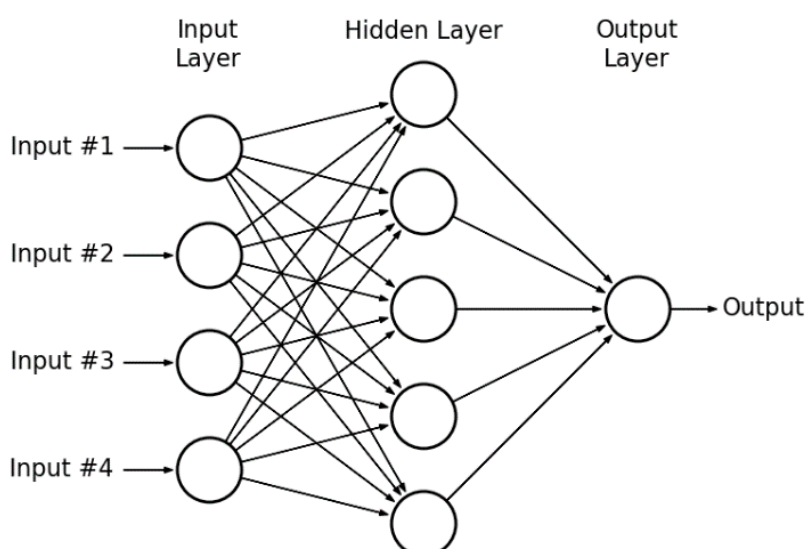
Cel ćwiczenia

Celem zadania była implementacja od podstaw sieci neuronowej typu MLP oraz przeprowadzenie eksperymentów klasyfikacji na zbiorze danych Wine Quality.

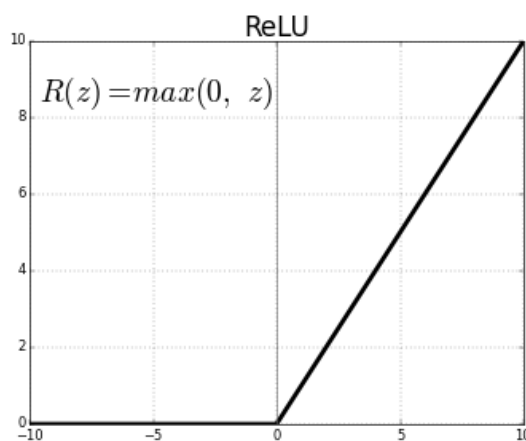
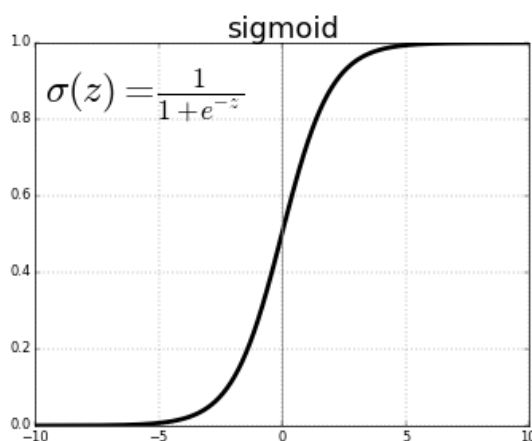
Zasada działania zaimplementowanej sieci (MLP)

Działanie zaimplementowanej sieci neuronowej typu MLP (Multi-Layer Perceptron) opiera się na iteracyjnym procesie uczenia nadzorowanego, który składa się z trzech kluczowych etapów:

1. **Propagacja w przód (Forward Pass):** Dane wejściowe są przetwarzane warstwa po warstwie. W każdym neuronie zachodzi **kombinacja liniowa** wejść i wag ($Z=Wx+b$), a następnie wynik poddawany jest **nieliniowej funkcji aktywacji** (ReLU lub Sigmoid). Pozwala to sieci na mapowanie złożonych zależności w danych.



-schemat budowy MLP



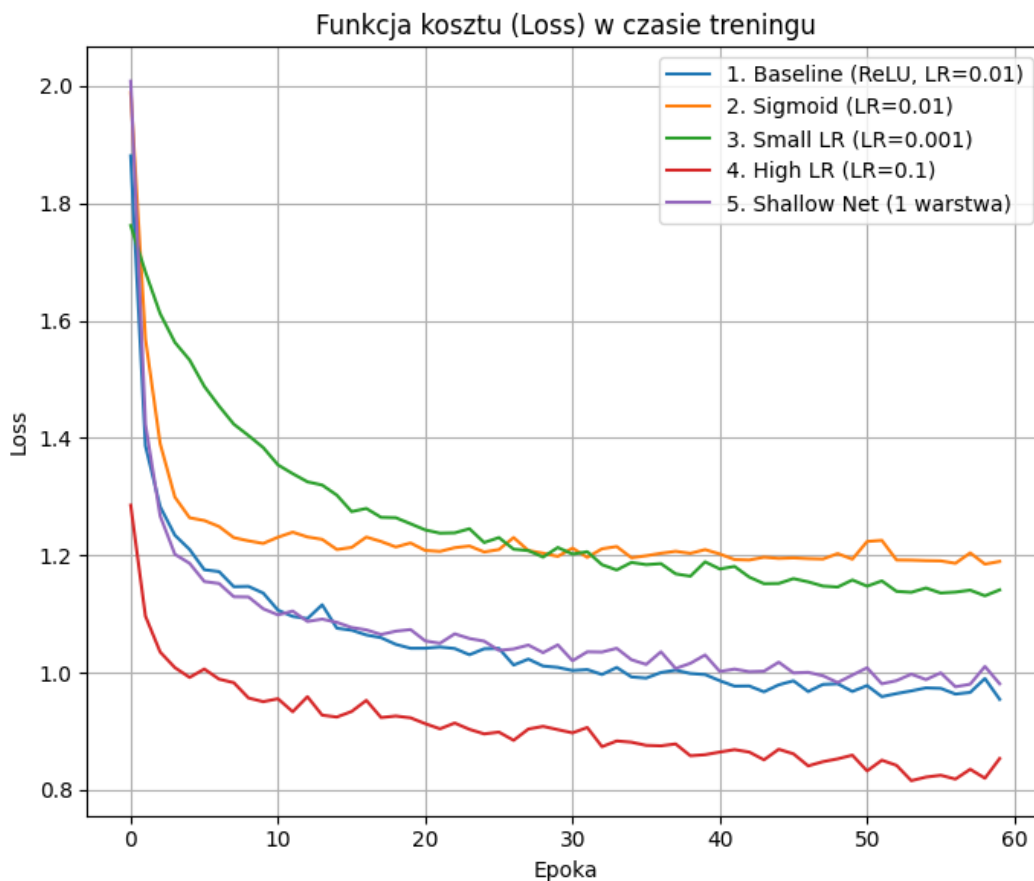
-wykresy ReLU i sigmoid

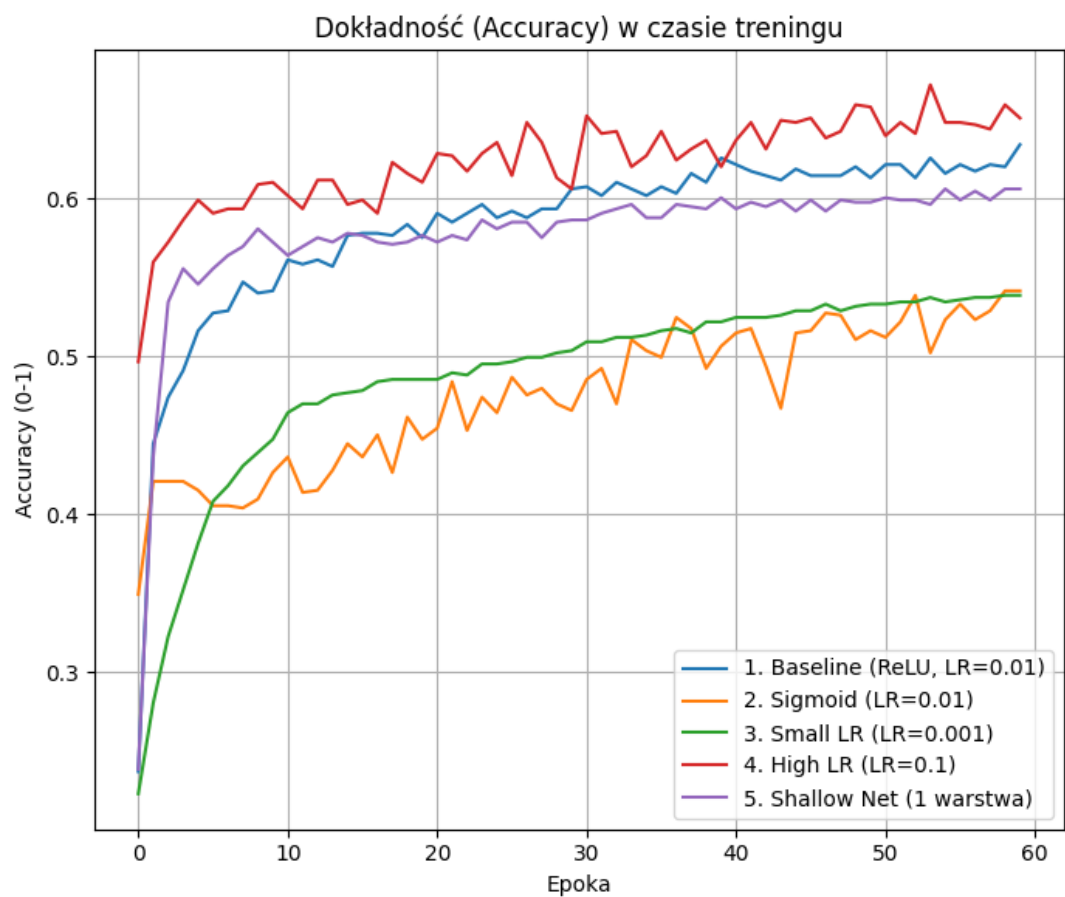
2. **Obliczenie błędu (Loss Calculation):** Ostatnia warstwa przekształca wyniki na rozkład prawdopodobieństwa za pomocą funkcji **Softmax**. Następnie funkcja kosztu **Entropii Krzyżowej (Cross-Entropy)** oblicza błąd, czyli numeryczną miarę różnicy między przewidywaniem sieci a rzeczywistą etykietą klasy.
3. **Propagacja wsteczna i aktualizacja wag (Backpropagation & SGD):** Algorytm oblicza **gradienty**, czyli pochodne cząstkowe funkcji błędu względem każdej wagi w sieci (informacja o tym, "w którą stronę" i jak mocno zmienić wagę). Na koniec algorytm **SGD** (Stochastic Gradient Descent) aktualizuje wagi, zmniejszając błąd sieci w kolejnych iteracjach.

Metodyka eksperymentów

Przeprowadzono serię eksperymentów mających na celu zbadanie wpływu hiperparametrów na jakość klasyfikacji.

- **Liczba epok:** 60
- **Rozmiar batcha:** 32.
- **Scenariusze testowe:**
 1. **Baseline:** Sieć głęboka (2 warstwy ukryte: 64, 32), aktywacja ReLU, LR=0.01.
 2. **Porównanie aktywacji:** Zamiana ReLU na Sigmoid przy tej samej architekturze.
 3. **Wpływ Learning Rate:** Testowanie małego kroku (LR=0.001) oraz dużego (LR=0.1).
 4. **Złożoność modelu:** Porównanie sieci głębokiej z płytką (1 warstwa ukryta).





Nazwa eksperymentu	Learning Rate	Aktywacja	Architektura	Accuracy końcowe
1. Baseline	0.01	ReLU	[64, 32]	61,99%
2. Sigmoid	0.01	Sigmoid	[64, 32]	52.03%
3. Small LR	0.001	ReLU	[64, 32]	55.96%
4. High LR	0.1	ReLU	[64, 32]	66.20%
5. Shallow Net	0.01	ReLU	[64]	61.57%

Ze względu na losową inicjalizację wag oraz stochastyczny charakter algorytmu SGD, pojedynczy pomiar może być niemiernodajny. Aby zweryfikować stabilność wyników, każdy eksperyment powtórzono 8 razy. Wyniki uśrednione przedstawiono w poniższej tabeli:

Eksperyment	Średnia Dokładność (Accuracy)	Odchylenie Standardowe (σ)	Min	Max
1. Baseline (ReLU, LR=0.01)	61.50%	$\pm 0.77\%$	60.59%	62.55%
2. Sigmoid (LR=0.01)	54.70%	$\pm 2.27\%$	51.61%	57.92%
3. Small LR (LR=0.001)	56.31%	$\pm 1.99\%$	53.30%	58.35%
4. High LR (LR=0.1)	65.73%	$\pm 1.96\%$	62.97%	68.58%
5. Shallow Net (1 warstwa)	61.15%	$\pm 0.81\%$	59.05%	62.69%

Wnioski

1. ReLU vs Sigmoid:

- Sieć z ReLU uczy się szybciej i osiągnąć lepszy wynik niż Sigmoid. Sigmoid w sieciach wielowarstwowych często cierpi na problem zanikającego gradientu (gradienty są bliskie zeru, więc wagi się nie zmieniają).

2. Wpływ Learning Rate:

- Mały LR (0.001): Uczenie powinno być bardzo wolne, a wykres Loss spadać łagodnie. Może nie zdążyć dojść do minimum w 60 epokach.
- Duży LR (0.1): Może dać najlepszy wynik, ale wykres Loss może być "poszarpany" (niestabilny), ponieważ sieć skacze wokół minimum.

3. Głębokość sieci (Deep vs Shallow):

- Sieć z dwiema warstwami ukrytymi ([64, 32]) zazwyczaj radzi sobie lepiej z nieliniowymi zależnościami w danych niż sieć płytka ([64]), choć przy prostych zbiorach danych różnica może być niewielka.

4. Analiza stabilności i powtarzalności wyników:

- **Najwyższa skuteczność (High LR):** Eksperyment nr 4 (High LR=0.1) osiągnął zdecydowanie najlepszą średnią dokładność (**65.73%**). Jednak odchylenie standardowe wynosi tu $\pm 1.96\%$, a rozrzut wyników to ponad 5 punktów procentowych (od 62.97% do 68.58%). Potwierdza to teorię, że

wysoki *Learning Rate* pozwala szybciej uciec z lokalnych minimów, ale proces uczenia jest mniej stabilny i bardziej losowy.

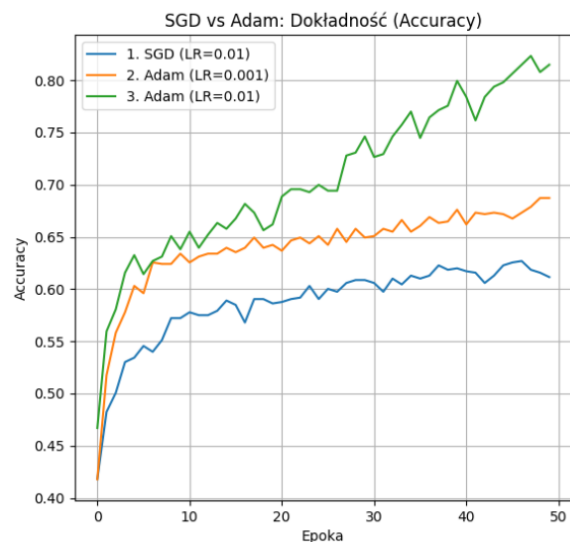
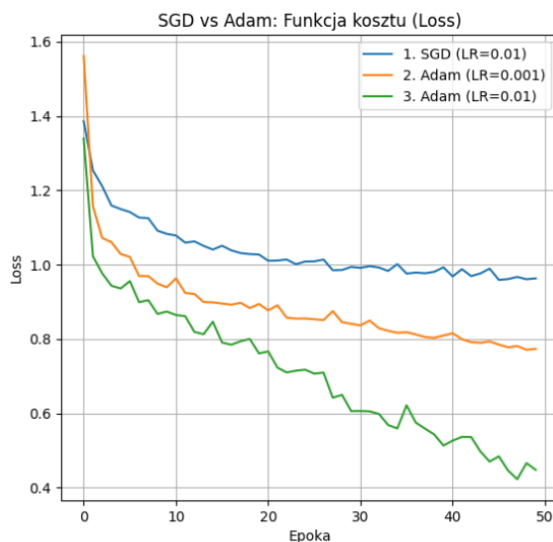
- **Najwyższa stabilność (Baseline i Shallow):** Zarówno Baseline ($\pm 0.77\%$) jak i sieć płytka ($\pm 0.81\%$) charakteryzują się bardzo małym odchyleniem standardowym. Oznacza to, że są to konfiguracje "bezpieczne" – niezależnie od losowej inicjalizacji wag, sieć prawie zawsze zbiega do tego samego poziomu ($\sim 61\%$).
- **Niestabilność Sigmoida:** Eksperyment z funkcją Sigmoid zanotował najgorszy średni wynik (**54.70%**) i największe odchylenie standardowe ($\pm 2.27\%$). Świadczy to o tym, że sieć z Sigmoidem ma trudności z uczeniem (problem zanikającego gradientu) i w niektórych przebiegach radzi sobie znacznie gorzej niż w innych, będąc bardzo wrażliwą na początkowy układ wag.
- **Wniosek końcowy:** Jeśli zależy nam na maksymalnym wyniku, należy wybrać **High LR** (mimo większej wariancji). Jeśli zależy nam na powtarzalności i stabilności procesu produkcyjnego, najlepszym wyborem jest konfiguracja **Baseline**.

Zadania dla chętnych

1. Algorytm Adam (Adaptive Moment Estimation) to zaawansowana metoda optymalizacji stochastycznej, która łączy w sobie zalety technik Momentum oraz RMSprop, adaptując współczynnik uczenia indywidualnie dla każdego parametru sieci neuronowej. W przeciwieństwie do klasycznego algorytmu SGD, który stosuje stały krok uczenia, Adam przechowuje dla każdej wagi dwie estymaty momentów: średnią kroczącą gradientów (pierwszy moment), która nadaje kierunkowi aktualizacji pęd, oraz średnią kroczącą kwadratów gradientów (drugi moment), która służy do normalizacji kroku uczenia. Dzięki temu wagi, których gradienty są duże i zmienne, aktualizowane są ostrożniej, natomiast te o rzadkich lub małych gradientach otrzymują silniejszy impuls do zmiany. Zaimplementowany algorytm wykorzystuje również mechanizm korekty obciążenia (bias correction), który kompensuje fakt, że momenty są inicjowane zerami, co zapobiega zbyt wolnemu uczeniu w początkowych epokach. Takie podejście sprawia, że Adam zazwyczaj osiąga zbieżność znacznie szybciej niż standardowy SGD, jest bardziej odporny na zaszumione dane i mniej wrażliwy na precyzyjny dobór początkowego współczynnika uczenia.

Najlepsze wyniki dla niego osiągnęliśmy dla $LR=0.01$, $\beta_1=0.9$, $\beta_2=0.999$ i $\epsilon=1e-8$. Najlepsze wartości osiąga już dla 60 epoki, później mamy do czynienia z przeuczeniem.

Algorytm i Konfiguracja	Średnia Dokładność ($\bar{x}$)	Niepewność	Min	Max
1. SGD (LR=0.01)	61.16%	$\pm 0.63\%$	60.31%	62.41%
2. Adam (LR=0.001)	68.41%	$\pm 1.29\%$	65.64%	70.41%
3. Adam (LR=0.01)	83.70%	$\pm 3.36\%$	78.82%	88.36%



2. Regularyzacja

- **Regularyzacja L1 (Lasso Regression)**

Zasada działania: L1 dodaje do funkcji kosztu (Loss) karę proporcjonalną do **wartości bezwzględnej wag**:

$$\text{Loss} = \text{LossData} + \lambda \sum |w|$$

Efekt (Selekcja Cech): L1 jest "brutalna". Jej matematyczna natura sprawia, że **ściąga mniej ważne wagi idealnie do zera**.

- **Co to robi?** Tworzy tzw. "rzadkie" (sparse) modele. Sieć sama decyduje, które wejścia są śmieciami i całkowicie je ignoruje (zeruje ich wagi).

- **Regularyzacja L2 (Ridge Regression / Weight Decay)**

Zasada działania: L2 dodaje do funkcji kosztu karę proporcjonalną do **kwadratu wag**:

$$\text{Loss} = \text{LossData} + \lambda \sum w^2$$

W implementacji często realizuje się to bezpośrednio jako **Weight Decay** (zanikanie wag) – w każdym kroku odejmujemy ułamek wagi (np. $w = w - \lambda \cdot w$).

Efekt (Wyglądanie Wag): L2 "nie lubi" dużych liczb. Karze bardzo mocno za jedną wielką wagę, a łagodnie za wiele małych wag.

- **Co to robi?** Sprawia, że wagi stają się małe i "rozmyte". Żaden pojedynczy neuron nie może zdominować decyzji sieci. Model staje się gładszy i mniej wrażliwy na drobne zmiany w danych wejściowych.

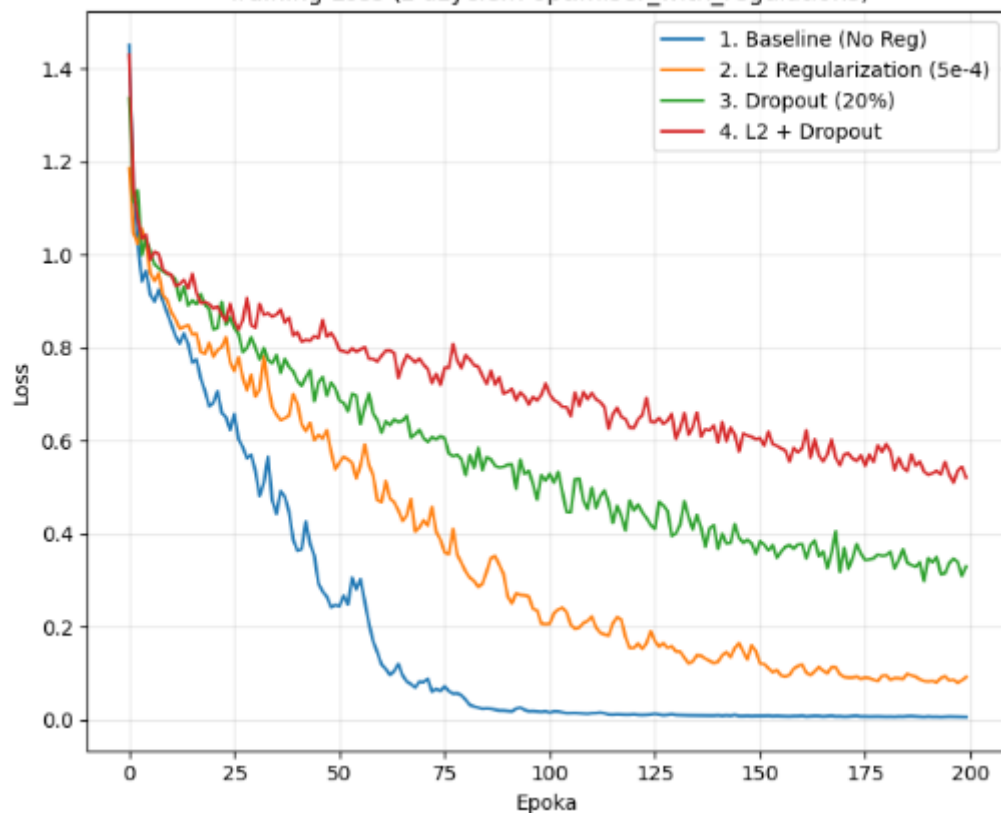
- **Dropout (Porzucanie)**

Zasada działania: To technika inna niż L1/L2, bo nie modyfikuje funkcji kosztu, ale samą **strukturę sieci**. Podczas treningu, w każdym przejściu (kroku), losowo "wyłączamy" (zerujemy) określony procent neuronów (np. 20%).

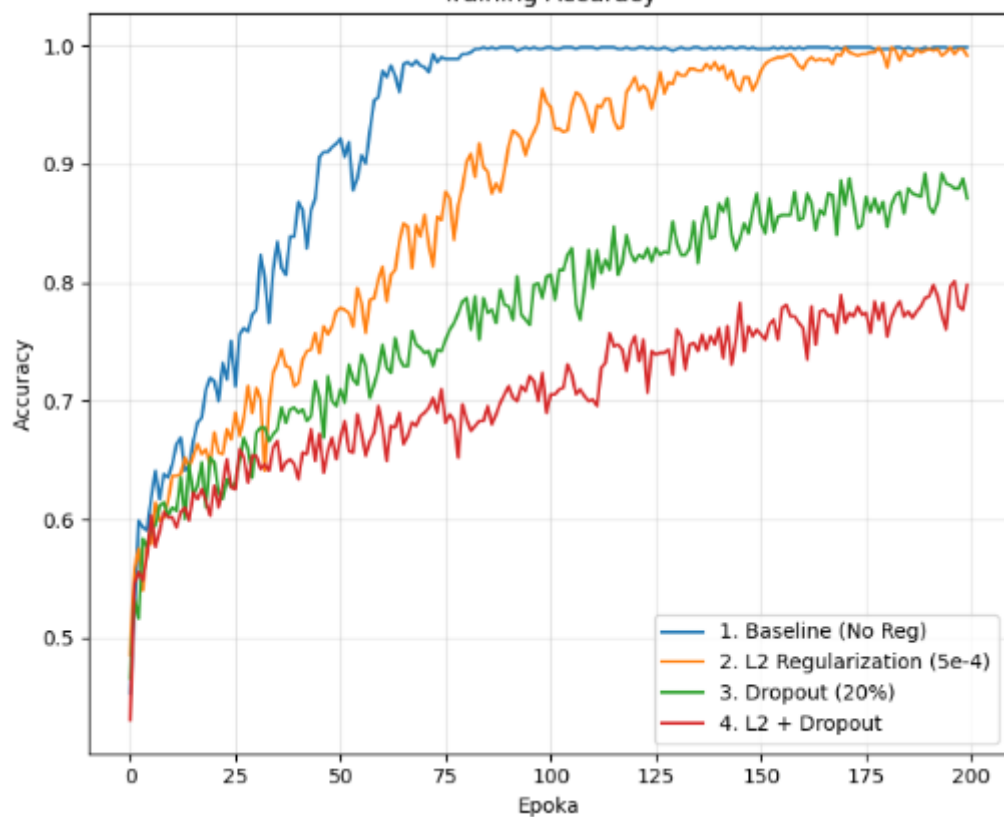
- **Co to robi?** Żaden neuron nie może polegać na obecności innego konkretnego neuronu (bo ten może zostać wyłączony). Każdy neuron musi nauczyć się radzić sobie samodzielnie i szukać użytecznych cech w różnych miejscach.
- **Interpretacja:** Dropout sprawia, że trenujemy tysiące "pod-sieci" naraz i uśredniamy ich wyniki. To działa jak **ensemble learning** (zespół ekspertów), co drastycznie poprawia generalizację.

Ważne: Dropout działa **tylko podczas treningu**. Podczas testowania/walidacji włączamy wszystkie neurony, ale skalujemy ich wagi, aby zachować balans sygnału.

Training Loss (z użyciem optimiser_with_regulations)



Training Accuracy



1. **Linia Niebieska (Baseline - Bez regularyzacji):**

- **Zachowanie:** Bardzo szybko osiąga niemal zerowy błąd (Loss) i 100% dokładności (Accuracy) na zbiorze treningowym.
- **Wniosek:** Model "nauczył się na pamięć" zbioru treningowego. To klasyczny objaw **overfittingu** (przeuczenia). Model nie ma żadnych ograniczeń, więc dopasowuje się idealnie do danych uczących.

2. **Linia Pomarańczowa (L2 Regularization):**

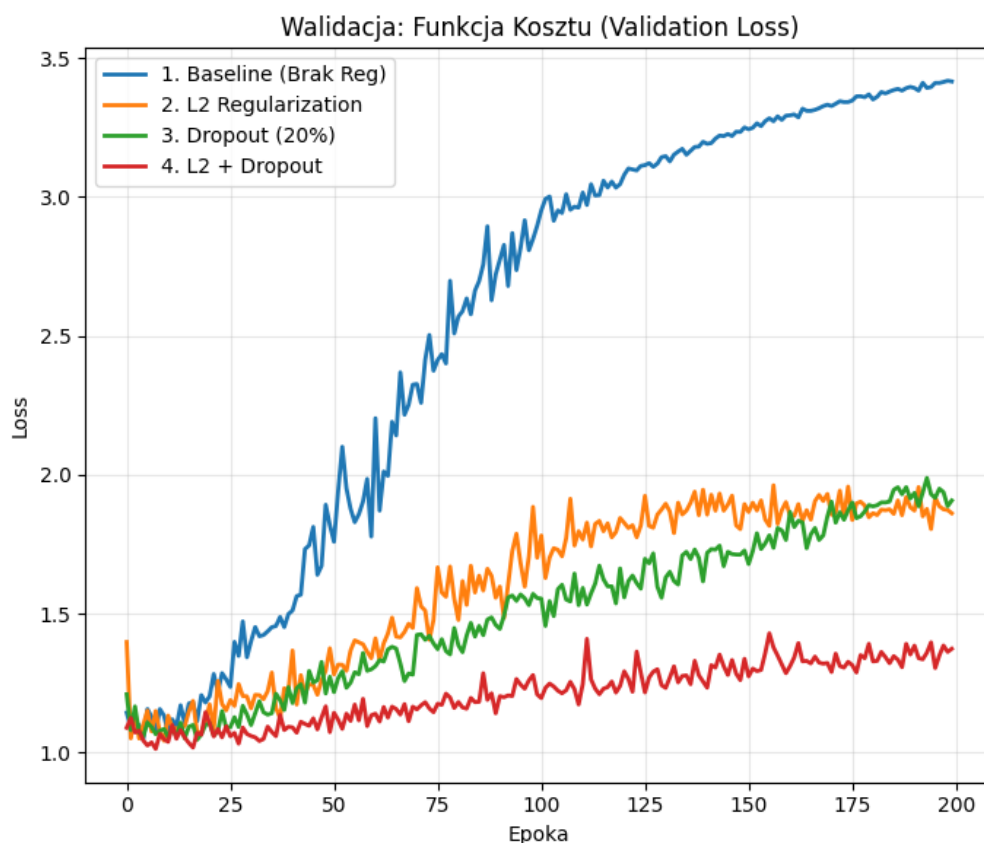
- **Zachowanie:** Uczy się wolniej niż Baseline. Loss spada wolniej, a dokładność rośnie łagodniej.
- **Wniosek:** Regularyzacja L2 ("Weight Decay") karze model za zbyt duże wagi, co utrudnia mu idealne dopasowanie się do szumu w danych treningowych. To pożądane zachowanie.

3. **Linia Zielona (Dropout 20%):**

- **Zachowanie:** Wykres jest poszarpany (ma "szum").
- **Wniosek:** To charakterystyczne dla Dropoutu. W każdej epoce losowo wyłączane są inne neurony, więc sieć ciągle uczy się w nieco innej konfiguracji. To sprawia, że trening jest trudniejszy (niższa dokładność na treningu), ale model staje się bardziej odporny.

4. **Linia Czerwona (L2 + Dropout):**

- **Zachowanie:** Najwolniejszy spadek funkcji kosztu i najniższa dokładność na zbiorze treningowym (ok. 80%).
- **Wniosek:** Połączenie obu metod nakłada na sieć bardzo silne ograniczenia. Sieć ma bardzo trudno "wkuć na pamięć" dane treningowe. Choć na wykresie *treningowym* wygląda to na najgorszy wynik, w rzeczywistości taki model często najlepiej radzi sobie na nowych danych (zbiór testowy), ponieważ został zmuszony do nauki ogólnych reguł, a nie szczegółów.



- **Niebieska linia (Baseline):** Bardzo szybko rośnie w górę. Jest to podręcznikowy przykład **overfittingu**. Sieć "wkuła" dane treningowe (tam loss dąży do 0), ale na danych walidacyjnych radzi sobie coraz gorzej.
- **Czerwona linia (L2 + Dropout):** Jest płaska i niska. To dowód, że implementacja działa. Regularyzacja powstrzymała sieć przed eksplozją błędów.